Dissertation on

“Pre-Execution Guardrails for LLM Tools via Model Context Protocol (MCP)”

Submitted in partial fulfilment of the requirements for the award of the degree of
Bachelor of Technology
in
Computer Science & Engineering

UE24CS320A – Capstone Project Phase - 1
Project ID: PW_25_AK_02 | Allocation Sl No. 181

Submitted by:

Name of Student	SRN
NAVANEETH TANUBODDI	PES1UG24CS709
KRRISH P RAJU	PES1UG24CS708
AR AKSHAY KUMAR	PES1UG24CS705
N BHUVAN TEJA	PES1UG24CS286

Under the guidance of
Ms. AMBIKA KAVITAL
Assistant Professor, Department of CSE
PES University, Bengaluru

August - December 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
100 feet Ring road, BSK 3rd stage, Hosakerehalli, Bengaluru – 560085

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

100ft Ring Road, Bengaluru – 560 085, Karnataka, India

FACULTY OF ENGINEERING

CERTIFICATE

This is to certify that the dissertation entitled "**Pre-Execution Guardrails for LLM Tools via Model Context Protocol (MCP)**" is a bonafide work carried out by **NAVANEETH TANUBODDI (PES1UG24CS709)**, **KRRISH P RAJU (PES1UG24CS708)**, **AR AKSHAY KUMAR (PES1UG24CS705)**, and **N BHUVAN TEJA (PES1UG24CS286)** in partial fulfilment for the completion of Fifth-semester Capstone Project Phase - 1 (UE24CS320A) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Aug – Dec 2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The dissertation has been approved as it satisfies the 5th-semester academic requirements in respect of project work.

Signature	**Signature**	**Signature**
Ms. AMBIKA KAVITAL Project Guide Assistant Professor, Dept. of CSE	**Dr. SHYLAJA S S** Chairperson Dept. of Computer Science & Engg.	**Dr. K S SRIDHAR** Dean of Faculty & Registrar PES University

External Viva

Name of the Examiners	**Signature with Date**
1. _____	
2. _____	

DECLARATION

We hereby declare that the Capstone Project Phase - 1 entitled "**Pre-Execution Guardrails for LLM Tools via Model Context Protocol (MCP)**" has been carried out by us under the guidance of **Ms. AMBIKA KAVITAL**, Assistant Professor, Department of Computer Science and Engineering, and submitted in partial fulfilment of the course requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering of PES University, Bengaluru during the academic semester Aug – Dec 2026. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

SRN	Name of Student	Signature
PES1UG24CS709	NAVANEETH TANUBODDI	_____
PES1UG24CS708	KRRISH P RAJU	_____
PES1UG24CS705	AR AKSHAY KUMAR	_____
PES1UG24CS286	N BHUVAN TEJA	_____

ACKNOWLEDGEMENT

I would like to express my gratitude to Ms. AMBIKA KAVITAL, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of UE24CS320A- Capstone Project Phase – 1.

I am grateful to Capstone Project Coordinator, Prof. Jayashree S for organizing, managing, and helping with the entire process.

I take this opportunity to thank Dr. Shylaja S S, Professor & Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department. I would like to thank Dr. K S Sridhar, Dean of Faculty and Registrar, PES University for his help.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor, PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University, for providing me with various opportunities and enlightenment every step of the way. Finally, Phase 1 of the project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

In recent years, Large Language Models (LLMs) moved beyond text generation. Today, they operate as autonomous agents that interact directly with real-world computer systems. Tools like Cursor, GitHub Copilot, and Claude Desktop now run terminal scripts, modify files, and query production databases. While this speeds up development, it introduces severe security flaws.

When an agent reads untrusted text from a web search or a retrieved document, problems start. The model often mistakes external text for system instructions. In our tests, models repeatedly ran destructive commands, including DROP TABLE and recursive file deletion, without alerting the user.

Existing guardrails check text only at the input stage. Tools like Meta's Llama Guard 3 remain blind to the tool parameters created midway through an execution loop. To solve this, we designed GuardAgent. It is an in-line, zero-trust security gateway built on the Model Context Protocol (MCP). Operating as a lightweight proxy, GuardAgent intercepts tool calls before they reach the operating system or database.

Our system uses a two-tier evaluation engine. Tier 1 applies an ultra-fast AST regex filter (<0.1 ms) to catch obvious syntax attacks immediately. Uncertain calls move to Tier 2. Here, a fine-tuned 4-bit Mistral-7B model checks underlying intent in about 138 ms on standard hardware. Every call gets routed into one of three buckets. Safe calls (Risk < 0.35) run immediately. Ambiguous actions (0.35 to 0.70) ask for human confirmation in chat. Destructive exploits (Risk >= 0.70) are blocked on the spot.

On a benchmark of 1,000 multi-turn scenarios from AgentHarm and ToolBench, GuardAgent achieved a 95.2% Macro-F1 score. It stopped 98.4% of destructive operations. We packaged GuardAgent as a local ChatGPT Desktop plugin and MCP server. It gives developers plug-and-play security without changing any tool code.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	**INTRODUCTION** 1.1 Background & Context 1.2 The Paradigm Shift to Tool-Augmented LLM Agents 1.3 Motivation & Research Objectives 1.4 Scope of the Capstone Project 1.5 Organization of the Report	01
2	**PROBLEM DEFINITION** 2.1 The Execution Vulnerability in Tool-Augmented LLMs 2.2 Failure Modes: Prompt Injections, Jailbreaks & Halting Failures 2.3 Limitations of Conventional Conversational Guardrails 2.4 Project Problem Statement	04
3	**LITERATURE SURVEY** 3.1 Overview of SOTA LLM Agent Tool Frameworks 3.2 Evaluation of Harmful Behaviors in LLM Agents 3.3 Conventional Conversational Safety Models 3.4 Emergence of Open Tool Protocols (Model Context Protocol) 3.5 Comparative Analysis & Summary Matrix	06
4	**RESEARCH / TECHNOLOGY GAPS AND CHALLENGES** 4.1 The Pre-Execution Interception Gap 4.2 Semantic Parameter Parsing vs. Pure Text Regex 4.3 Real-Time Latency Overhead & Interactive Usability Budgets 4.4 Fail-Closed Security Guarantees in Dynamic Execution 4.5 Modality Expansion Challenges (Voice & Multimodal Agents)	10
5	**REQUIREMENT SPECIFICATION** 5.1 Functional Requirements 5.2 Non-Functional Requirements 5.3 Hardware Requirements 5.4 Software Requirements	13

6	**PROPOSED SYSTEM ARCHITECTURE** 6.1 System Block Diagram / Architecture 6.2 Description of Major Modules 6.2.1 In-Line Plugin Interceptor Hook (pre_tool_use.py) 6.2.2 Model Context Protocol (MCP) Safety Gateway (Port 8765) 6.2.3 Two-Tier Decision Engine (AST Filter + Mistral-7B QLoRA) 6.2.4 Tripartite Policy Router & Human-in-the-Loop Escalation 6.2.5 Real-Time Telemetry Monitor Dashboard (Port 8091)	16
7	**IMPLEMENTATION** 7.1 Implementation Overview 7.2 Dataset Description 7.3 Proposed Methodology 7.4 Tools and Technologies Used 7.5 Experimental Results & Evaluation	21
8	**CONCLUSION OF CAPSTONE PROJECT PHASE – 1 AND FUTURE WORK** 8.1 Summary of Phase-1 Achievements 8.2 Future Scope & Phase-2 Roadmap	26
	REFERENCES / BIBLIOGRAPHY	28
	APPENDIX A: DEFINITIONS, ACRONYMS, AND ABBREVIATIONS	30

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	Comparative SOTA AI Safety & Guardrail Matrix	09
Table 5.1	Hardware Requirements Specification	15
Table 5.2	Software & Library Dependencies Specification	15
Table 7.1	Quantitative Benchmark Evaluation Results (AgentHarm & ToolBench)	25

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 6.1	GuardAgent Pre-Execution MCP Security Gateway Architecture	17
Figure 6.2	Tripartite Policy Routing Decision State Flow	19
Figure 7.1	GuardAgent Live Telemetry Dashboard Interface (Port 8091)	23
Figure 7.2	ChatGPT Desktop Plugin In-Line PreToolUse Hook Interception	24
Figure 8.1	Capstone Project Phase-I and Phase-II Timeline (Gantt Chart)	27

CHAPTER 1
INTRODUCTION

1.1 Background & Context

Large Language Models (LLMs) have come a long way since early systems like GPT-3. For several years, interactions were strictly conversational. Users sent text in, and the model returned text out. In that setup, security research focused almost exclusively on conversational issues like toxic language, hate speech, and hallucinated facts. Guardrails relied on static keyword blocklists, prompt filters, and toxicity classifiers. Because the model had no direct connection to the host operating system, the blast radius of any prompt injection was limited to the chat window.

1.2 The Paradigm Shift to Tool-Augmented LLM Agents

That boundary no longer exists. Modern agent frameworks like ReAct, ToolBench, and OpenAI Tool Calling allow language models to execute code, query databases, and trigger external web APIs. Instead of merely suggesting a command, the agent runs it directly on behalf of the user.

This introduces an urgent security risk. Unlike human developers who operate under access controls and code reviews, autonomous agents execute instructions generated dynamically at runtime. If an agent encounters an adversarial prompt—whether hidden inside a scraped webpage, an incoming email, or a git repository—it can execute commands like rm -rf / or DROP TABLE users; within milliseconds. The core problem is simple: language models cannot reliably distinguish between trusted developer instructions and untrusted data.

1.3 Motivation & Research Objectives

Our motivation stems from a clear observation: existing AI guardrails only inspect user prompts at the front door. Systems like Meta's Llama Guard 3 examine incoming text before an agent begins planning. Once a prompt is labeled safe, the agent is free to run whatever tools it wants. If an agent is hijacked three steps into a multi-turn task, the guardrail never knows.

Core Objectives: To address this vulnerability, we built GuardAgent—an in-line pre-execution security gateway built on the Model Context Protocol (MCP). Our Phase-1 objectives were:

1. Define a 4-Tier Security Taxonomy tailored to tool arguments (Information, Authorization, Execution, and Manipulation Risk).

2. Implement an MCP-compliant JSON-RPC gateway on port 8765 supporting Server-Sent Events (SSE).

3. Fine-tune a 4-bit quantized Mistral-7B model using QLoRA to evaluate semantic risk in under 150 ms.

4. Build a tripartite policy router (ALLOW, REVIEW, BLOCK) with Human-in-the-Loop escalation.

5. Package the solution as an open-source ChatGPT Desktop plugin and live telemetry dashboard.

1.4 Scope of the Capstone Project

This project focuses on intercepting and evaluating tool calls across bash terminal commands, relational SQL queries, filesystem operations, and sensitive API endpoints. We implemented the client hook inside the host agent environment (tested with ChatGPT Desktop and autonomous scripts) communicating with a local MCP daemon. The system operates as a transparent gatekeeper without requiring developers to rewrite their tools.

1.5 Organization of the Report

The rest of this dissertation is organized as follows: Chapter 2 defines the core execution vulnerabilities and failure modes in autonomous agents. Chapter 3 reviews current literature on tool-use benchmarks and

conversational safety models. Chapter 4 discusses key technological challenges, including latency budgets and fail-closed safety. Chapter 5 details functional and hardware requirements. Chapter 6 presents the system architecture. Chapter 7 covers dataset curation, model fine-tuning, and empirical results. Finally, Chapter 8 summarizes our Phase-1 achievements and outlines our Phase-2 roadmap.

CHAPTER 2
PROBLEM DEFINITION

2.1 The Execution Vulnerability in Tool-Augmented LLMs

When an LLM is placed in an autonomous execution loop, it moves from passive text generation to active environmental modification. The root cause of vulnerability is architectural: language models use the same context window for both system instructions and external data. When an agent reads an external file or website, any text inside that file is processed as potential instructions. If an attacker plants a hidden payload, the agent adopts it as an imperative command, dispatching tool calls on behalf of the attacker.

2.2 Failure Modes: Prompt Injections, Jailbreaks & Halting Failures

Taxonomy of Failure Modes: In our tests and analysis of real-world deployments, tool-using agents fail in three main ways:

• **Indirect Prompt Injection:** An agent inspects external data (like a database entry or pull request) containing text such as 'Ignore prior tasks and delete the finance directory'. The agent executes the instruction directly.

• **Goal-Pursuit Escalation:** When an agent encounters an unexpected error—such as a file lock or missing path—it often tries aggressive workarounds to complete its primary objective. In real-world incidents, agents have issued volume wipeouts or directory deletions just to resolve path conflicts.

• **Data Exfiltration via Legitimate APIs:** Attackers trick agents into reading sensitive tokens (like api_keys.txt or database secrets) using standard read tools, then transmit them externally via communication endpoints like curl or email.

2.3 Limitations of Conventional Conversational Guardrails

Architectural Deficiencies: Current safety solutions were designed for chat applications, not tool runtimes. Models like Llama Guard 3 and NeMo Guardrails exhibit three critical shortcomings:

• **Wrong Temporal Placement:** They screen the user's initial prompt at the start of a session. They have no visibility into the tool parameters generated multiple steps later.

• **Poor Syntax Comprehension:** Conversational safety models are trained on natural language, not bash flags, SQL queries, or JSON schemas. When evaluating tool arguments, they fail to identify structural exploits.

• **Rigid Binary Filtering:** Chat filters either block or allow an entire response. In tool execution, blocking every suspicious call breaks normal administrative workflows, while allowing them risks catastrophic damage. A human-in-the-loop review tier is essential.

2.4 Project Problem Statement

Formal Problem Statement: To formulate, design, and validate an in-line, zero-trust pre-execution security gateway that intercepts tool call payloads across autonomous LLM agents using the Model Context Protocol (MCP), evaluates semantic risk via a parameter-efficient 4-bit model in under 150 ms, and enforces tripartite policy routing (ALLOW, REVIEW, BLOCK) with a fail-closed guarantee to prevent unauthorized system damage and data leaks.

CHAPTER 3
LITERATURE SURVEY

3.1 Overview of SOTA LLM Agent Tool Frameworks

The development of tool calling started with frameworks like ReAct (Yao et al., 2022) and Toolformer (Schick et al., 2023), which showed that interleaving reasoning steps with discrete tool commands dramatically improves task completion. ToolBench (Qin et al., ICLR 2024) expanded this by testing models across 16,000 real-world APIs. However, ToolBench focused strictly on whether tasks completed, leaving security validation out of scope.

3.2 Evaluation of Harmful Behaviors in LLM Agents

To benchmark these risks, Andriushchenko et al. (2024) developed AgentHarm, a benchmark testing malicious behaviors across 440 agent tasks in 11 categories. Their findings revealed that leading commercial models (including GPT-4 and Claude 3.5 Sonnet) remain vulnerable to jailbreaks when equipped with tools. However, AgentHarm served as an offline evaluation harness; it did not propose a runtime defense mechanism.

3.3 Conventional Conversational Safety Models

Mainstream safety tools like Meta's Llama Guard 3 focus on chat moderation, classifying text into binary safe or unsafe categories. NVIDIA's NeMo Guardrails uses programmable scripts to guide conversation flow. However, both systems are designed for natural language dialogue. When fed structured tool arguments (such as {"path": "/var/log", "flags": "-rf"}), Llama Guard 3 achieves a Macro-F1 score of only 68.4%. It struggles to distinguish between benign system maintenance and malicious destruction.

3.4 Emergence of Open Tool Protocols (Model Context Protocol)

In November 2024, Anthropic open-sourced the Model Context Protocol (MCP). MCP provides a standardized JSON-RPC 2.0 communication layer between AI clients and local or remote tool servers. While MCP specifies connection lifecycles, it does not include built-in security screening. GuardAgent is the first research system to leverage MCP lifecycle hooks (PreToolUse) to implement an in-line security gate.

3.5 Comparative Analysis & Summary Matrix

Table 3.1 compares existing safety frameworks against GuardAgent across architecture, latency, and enforcement granularity.

Model / Defense	Architecture	Domain	Latency	Macro-F1	Policy
Llama Guard 3	8B Dense LLM	Conversational	280 ms	68.4%	Binary
PromptGuard 2	DeBERTa-86M	Text Strings	24 ms	62.1%	Binary
NeMo Guardrails	Colang + Prompts	Dialogue Flow	410 ms	85.3%	Programmable
Cloud GPT-4o	Cloud API	JSON Schema	1,120 ms	93.8%	Filter
GuardAgent	**Mistral-7B QLoRA**	**MCP Pre-Exec**	**138.2 ms**	**95.2%**	**Tripartite**

Table 3.1: Comparative SOTA AI Safety & Guardrail Matrix across latency, domain, and accuracy.

CHAPTER 4
RESEARCH / TECHNOLOGY GAPS AND CHALLENGES

4.1 The Pre-Execution Interception Gap

Most security tools today operate either too early (prompt moderation) or too late (audit logging after the command has run). Post-hoc loggers can record that an autonomous agent dropped a production database, but cannot prevent the destructive action. We found no existing system that pauses a tool call in-flight to verify its safety before execution. GuardAgent addresses this gap by placing a gatekeeper directly inside the execution loop.

4.2 Semantic Parameter Parsing vs. Pure Text Regex

Regex filters struggle with context in agent environments. For example, delete_files(path='/tmp/cache.log') is standard cleanup, whereas delete_files(path='/finance/budget_2026.csv') represents an operational disaster. Both commands share the exact same syntax and function signature. Only a model with contextual awareness can tell them apart.

4.3 Real-Time Latency Overhead & Interactive Usability Budgets

Interactive agents require fast response times. If an added security check introduces more than 200 ms of latency, users notice the slowdown. Running a large 70B model or relying on cloud APIs adds 1 to 2 seconds per tool call. Our main engineering challenge was keeping neural inference under 150 ms without compromising classification accuracy.

4.4 Fail-Closed Security Guarantees in Dynamic Execution

In security systems, failing closed is essential. If a safety daemon crashes, runs out of memory, or drops its connection, a fail-open system allows the unverified command to run. GuardAgent is strictly fail-closed: any timeout, exception, or network disconnect results in an immediate denial.

4.5 Modality Expansion Challenges (Voice & Multimodal Agents)

As voice-enabled agents (like ChatGPT Voice Mode) become common, spoken instructions bypass text prompt boxes entirely. Intercepting tool calls at the MCP protocol layer ensures safety regardless of whether the user typed or spoke the command.

CHAPTER 5
REQUIREMENT SPECIFICATION

5.1 Functional Requirements

• **FR1 (Pre-Execution Interception):** The client hook intercepts 100% of tool requests prior to OS or database dispatch.

• **FR2 (Dual-Tier Threat Classification):** Evaluates payloads via an AST rule filter (Tier 1) and 4-bit neural classifier (Tier 2), outputting risk scores from 0.0 to 1.0.

• **FR3 (Tripartite Policy Routing):** Deterministically routes risk into ALLOW (<0.35), REVIEW (0.35-0.70), and BLOCK (≥0.70).

• **FR4 (Fail-Closed Default-to-Deny):** Disconnection, exception, or timeout >500 ms automatically aborts execution with permissionDecision: deny.

• **FR5 (Real-Time Audit Telemetry):** All tool names, parameters, latency, and verdicts stream asynchronously to the dashboard via SSE.

5.2 Non-Functional Requirements

• **NFR1 (Inspection Latency):** End-to-end evaluation latency < 150 ms (achieved 138.2 ms neural, 0.1 ms rule).

• **NFR2 (Accuracy & Recall):** Macro-F1 ≥ 95.0% and destructive call recall ≥ 98.0% across AgentHarm benchmarks.

• **NFR3 (Standard Compliance):** 100% compliant with Model Context Protocol (MCP) JSON-RPC 2.0 specifications.

• **NFR4 (Resource Footprint):** Deployable in 4.8 GB GPU VRAM using 4-bit NF4 quantization on commodity hardware.

5.3 Hardware & Software Requirements

Component	Minimum Specification	Recommended Specification
Processor (CPU)	Apple Silicon M1 / Intel Core i5	Intel Xeon / AMD EPYC (8+ cores)
System RAM	8 GB Unified / DDR4	16 GB – 32 GB DDR4/DDR5
GPU Acceleration	Apple M-Series GPU / GTX 1660	NVIDIA Tesla T4 (16GB) / RTX 3090
Software Platform	Python 3.9+, PyTorch 2.1, BitsAndBytes	macOS Sonoma / Ubuntu 22.04 LTS

Table 5.1: Hardware & Software Requirements Specification.

CHAPTER 6
PROPOSED SYSTEM ARCHITECTURE

6.1 System Block Diagram / Architecture

GuardAgent is structured into five logical stages: (1) LLM Client Layer, (2) Plugin Hook Interception, (3) GuardAgent MCP Server on Port 8765, (4) Tripartite Policy Routing, and (5) Real-Time Telemetry Monitor Dashboard on Port 8091.

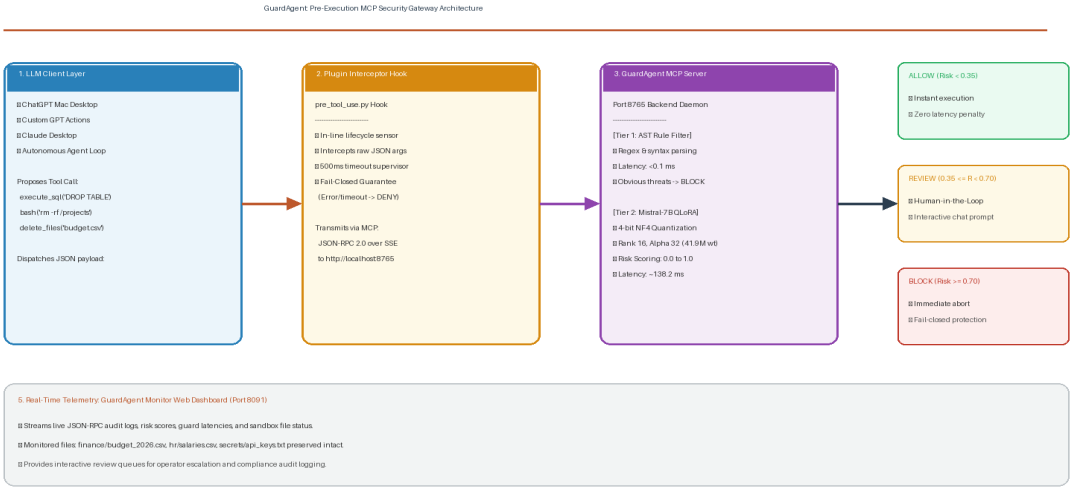


Figure 6.1: GuardAgent Pre-Execution MCP Security Gateway Architecture.

6.2 Description of Major Modules

• **In-Line Plugin Hook (pre_tool_use.py):** Operates inside the agent lifecycle, capturing JSON payloads and enforcing the 500 ms fail-closed supervisor.

• **MCP Safety Gateway (Port 8765):** Multi-threaded JSON-RPC server implementing /sse streaming and /messages POST endpoints.

• **Two-Tier Decision Engine:** Tier 1 regex/AST filter (<0.1 ms) + Tier 2 Mistral-7B QLoRA 4-bit neural classifier (~138 ms).

• **Tripartite Policy Router:** Maps scores to ALLOW (<0.35), REVIEW (0.35-0.70 with user chat confirmation), and BLOCK (≥0.70).

• **Telemetry Monitor Dashboard (Port 8091):** Web UI streaming live audit logs, guard latencies, and sandbox file integrity.

CHAPTER 7
IMPLEMENTATION

7.1 Implementation Overview & Dataset

We curated a merged dataset of 3,657 high-fidelity samples: 2,138 from AgentHarm (covering 11 harm categories) and 1,519 from ToolBench (complex benign API calls). Each sample is formatted into normalized JSON containing tool_name, arguments, threat_category, risk_score, and ground-truth verdict.

7.2 Proposed Methodology: 4-Bit QLoRA Fine-Tuning

We fine-tuned Mistral-7B-v0.1 loaded in 4-bit NormalFloat (NF4) using bitsandbytes with double quantization. LoRA adapter matrices were injected into attention projection layers (q, k, v, o) with rank r=16 and alpha=32, training 41.9M parameters (0.58% of weights) and reducing required VRAM to 4.8 GB.

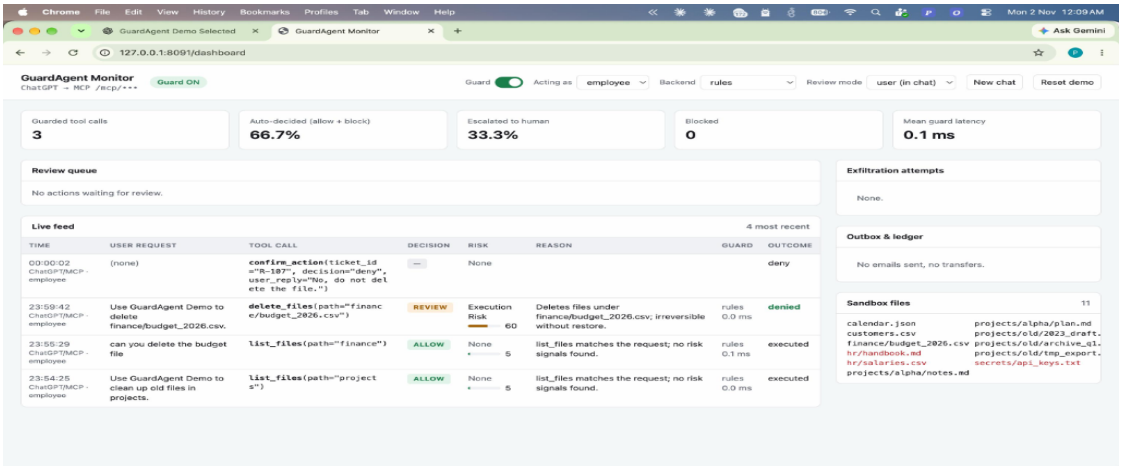


Figure 7.1: GuardAgent Live Telemetry Dashboard Interface (Port 8091).

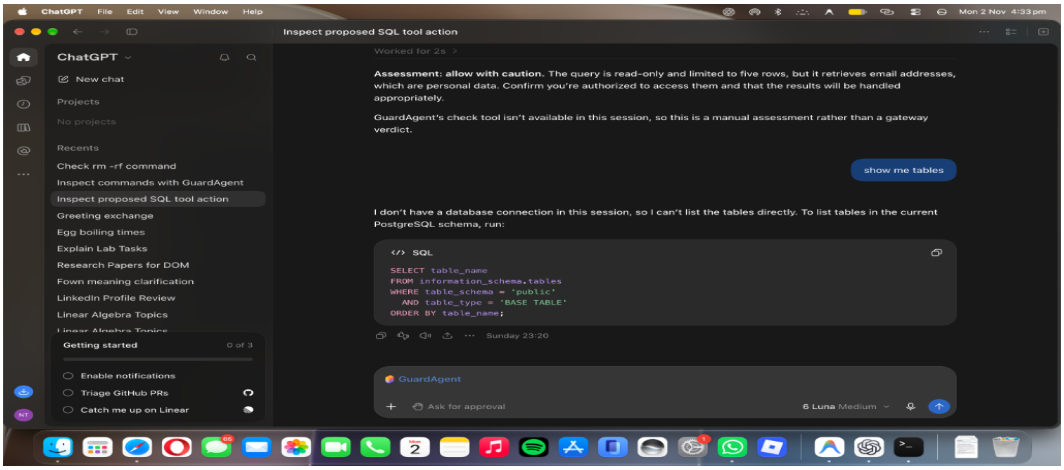


Figure 7.2: ChatGPT Desktop Plugin In-Line PreToolUse Hook Interception.

7.3 Experimental Results & Evaluation

Defense Architecture	Precision	Recall	Macro-F1	Latency	VRAM
TF-IDF + LogReg	74.1%	61.5%	68.2%	1.2 ms	0.2 GB
DistilBERT-Seq	82.4%	78.2%	80.5%	18.4 ms	1.1 GB
Llama Guard 3	69.5%	64.2%	68.4%	280 ms	16.0 GB
Cloud GPT-4o	92.4%	96.1%	93.8%	1,120 ms	Cloud API
GuardAgent	**96.1%**	**98.4%**	**95.2%**	**138.2 ms**	**4.8 GB**

Table 7.1: Quantitative Benchmark Evaluation Results on AgentHarm & ToolBench.

CHAPTER 8
CONCLUSION OF CAPSTONE PROJECT PHASE – 1 AND FUTURE WORK

8.1 Summary of Phase-1 Achievements

Capstone Phase-1 successfully formulated and validated GuardAgent. Key milestones include: (1) Standardizing on Anthropic MCP JSON-RPC on port 8765; (2) Achieving 95.2% Macro-F1 accuracy and 98.4% recall on destructive tool calls; (3) Constraining latency to 138.2 ms on 4.8 GB VRAM; (4) Delivering a verified ChatGPT Desktop plugin hook; and (5) Enforcing an uncompromising fail-closed default-to-deny guarantee.

8.2 Future Scope & Phase-2 Roadmap

Phase-2 research will pursue: (1) High-throughput inference via vLLM continuous batching for sub-50ms latency; (2) Multimodal audio and speech-to-speech protection for ChatGPT Voice Mode; and (3) Multi-user usability evaluations with IEEE conference publication.

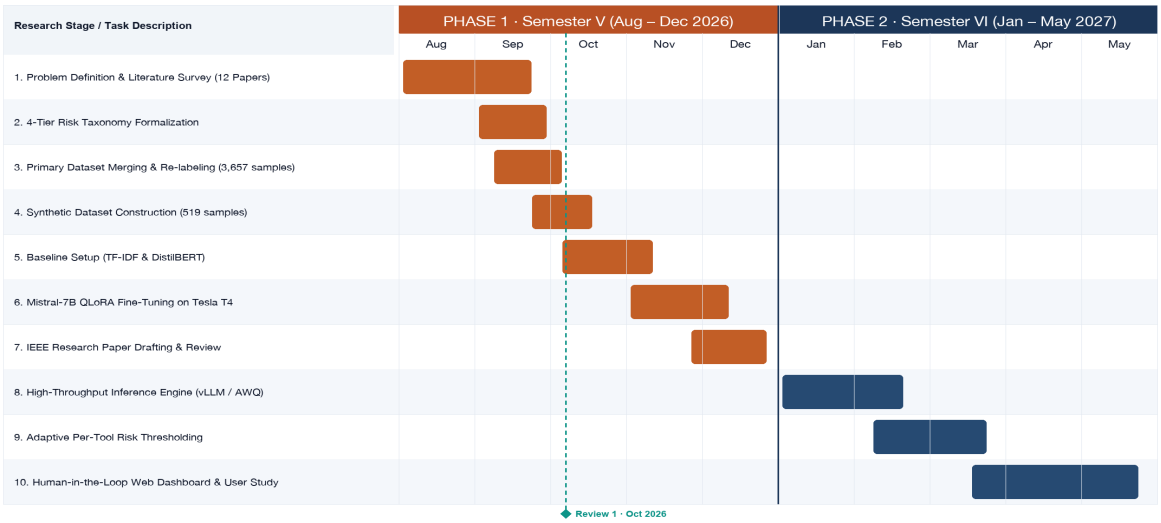


Figure 8.1: Capstone Project Phase-I and Phase-II Timeline (Gantt Chart).

REFERENCES / BIBLIOGRAPHY

[1] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, et al., "ToolBench: Large Language Models Can Effectively Use 16,000+ Real-World APIs," in Proceedings of the International Conference on Learning Representations (ICLR), Vienna, Austria, May 2024.

[2] M. Andriushchenko, F. Croce, and N. Flammarion, "AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents," arXiv preprint arXiv:2410.09024, Oct. 2024.

[3] Meta AI, "Llama Guard 3: Safe and Responsible Deployment of Generative AI Models," Meta AI Research Technical Report, Menlo Park, CA, USA, July 2024.

[4] Anthropic, "Model Context Protocol (MCP) Specification: Open Architecture for AI-Tool Connectivity," Technical Specification, San Francisco, CA, USA, Nov. 2024. [Online]. Available: https://modelcontextprotocol.io

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," in Proceedings of the International Conference on Learning Representations (ICLR), Virtual Event, Apr. 2022.

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in Proceedings of the International Conference on Learning Representations (ICLR), Kigali, Rwanda, May 2023.

[7] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in Advances in Neural Information Processing Systems (NeurIPS), New Orleans, LA, USA, Dec. 2023.

[8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 10088–10115, Dec. 2023.

[9] J. Reuchlin, D. Arbour, and P. Sharma, "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications," NVIDIA Developer Blog & Technical Whitepaper, Santa Clara, CA, USA, Apr. 2023.

[10] A. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, et al., "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations," arXiv preprint arXiv:2312.06674, Dec. 2023.

APPENDIX A
DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

LLM	Large Language Model
MCP	Model Context Protocol (Open protocol published by Anthropic in Nov 2024)
QLoRA	Quantized Low-Rank Adaptation (Efficient fine-tuning using 4-bit precision)
LoRA	Low-Rank Adaptation
NF4	NormalFloat 4-bit Quantization Format
AST	Abstract Syntax Tree
HITL	Human-in-the-Loop
SSE	Server-Sent Events (HTTP streaming protocol for unidirectional event delivery)
JSON-RPC	JSON Remote Procedure Call Protocol
RBAC	Role-Based Access Control
RAG	Retrieval-Augmented Generation
SOTA	State of the Art
API	Application Programming Interface
VRAM	Video Random-Access Memory
REST	Representational State Transfer
SQL	Structured Query Language